

MANUTENÇÃO:

CORREÇÃO DE BUGS:

Durante essa etapa, a equipe identificou e corrigiu um problema da interface, que não executava em outras máquinas, porém, ao baixar as bibliotecas necessárias no VScode conseguimos corrigir esse bug.

```
from abc import ABC, abstractmethod
import re
import sqlite3
import os
from typing import Optional, List, Tuple
import hashlib
import secrets
```



Outra parte que tivemos que fazer manutenção, foi a parte de email, posteriormente conversamos com o cliente que pediu para removermos essa função do admin e dos clientes pois achou desnecessário, já que o contato é feito pelo celular da pessoa. Aqui está o código antigo em que removemos o email do class usuario:

```

class Usuario:
    """
    Classe simples para representar usuário com nome, email e senha (armazenada como hash d
    Uso: u = Usuario("Nome", "email"); u.set_password("senha"); academia.add_usuario_simple
    """
    def __init__(self, nome: str, email: str, pwd_hash: str = None, salt: str = None):
        self.nome = (nome or "").strip()
        self.email = (email or "").strip().lower()
        self.pwd_hash = pwd_hash # hex string
        self.salt = salt # hex string

```

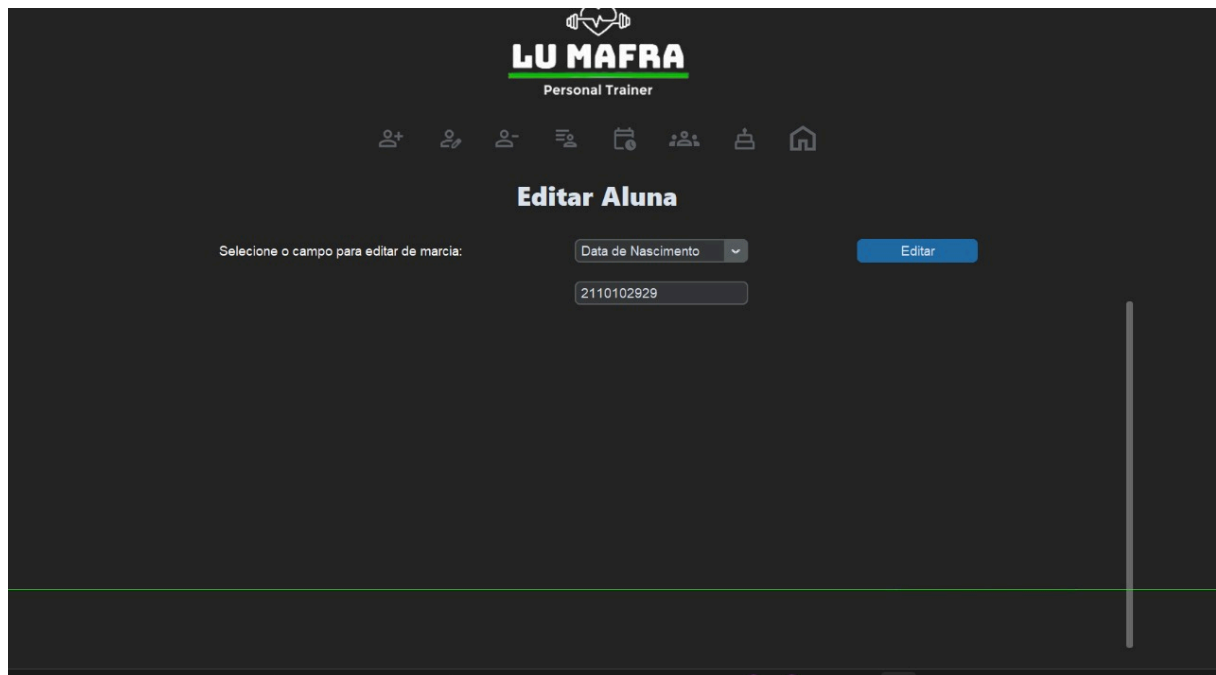
```

# ===== Métodos simples de gerenciamento de usuários =====
def add_usuario_simples(self, usuario: Usuario) -> bool:
    """
    Tenta inserir usuário. Retorna True se inseriu com sucesso, False se email já
    """
    with self._connect() as conn:
        cur = conn.cursor()
        try:
            cur.execute('''
                INSERT INTO usuarios (nome, email, pwd_hash, salt) VALUES (?, ?, ?
            ''', usuario.to_db_tuple())
            conn.commit()
            return True
        except sqlite3.IntegrityError:
            return False

def autenticar_usuario_simples(self, email: str, senha: str) -> Optional[Usuario]:
    """
    Retorna Usuario se autenticação OK, senão None.
    """
    with self._connect() as conn:
        cur = conn.cursor()
        cur.execute('SELECT nome, email, pwd_hash, salt FROM usuarios WHERE email
        row = cur.fetchone()
        if not row:
            return None
        user = Usuario.from_db_row(row)
        if user and user.check_password(senha):
            return user
        return None

```

Ainda por cima, havia um bug de formatação: ao inserir os dados, eles não eram formatados da maneira esperada:



Para que o bug pudesse ser resolvido, foi necessário as seguintes modificações no código (antes e depois, respectivamente):

```
def atualizar_mascara(valor):
    valor = entryInfo.get()
    label_nova_info.unbind("<KeyRelease>")

    if valor == "Data de Nascimento":
        label_nova_info.bind("<KeyRelease>", formatar_data)
    elif valor == "CPF":
        label_nova_info.bind("<KeyRelease>", formatar_cpf)
    elif valor == "Celular":
        label_nova_info.bind("<KeyRelease>", formatar_telefone)

    entryInfo.bind("<<ComboboxSelected>>", atualizar_mascara)

    botoaoPesquisar.configure(text="Editar", command= lambda: exibeTexto())
```

```
def atualizar_mascara(valor):
    try:
        label_nova_info.unbind("<KeyRelease>")
    except Exception:
        pass

    if valor == "Data de Nascimento":
        label_nova_info.bind("<KeyRelease>", formatar_data)
    elif valor == "CPF":
        label_nova_info.bind("<KeyRelease>", formatar_cpf)
    elif valor == "Celular":
        label_nova_info.bind("<KeyRelease>", formatar_telefone)
    else:
        pass

    entryInfo.configure(command=lambda v=None: atualizar_mascara(entryInfo.get()))

    botoaoPesquisar.configure(text="Editar", command= lambda: exibeTexto())
```

Durante o projeto, houve um bug de design na área de buscar alunas deste código antigo que tinha um scroll de mouse não era removível, então no código novo isso foi corrigido, aqui o código antigo em que o scroll permanecia:

```
def buscar_alunas():
    nonlocal entryacao

    for widget in widgetEditor.winfo_children():
        if isinstance(widget, ctk.CTkScrollableFrame):
            widget.destroy()

    label_status.configure(text="")
    label_status.grid_forget()

    scroll.place(relx=0.5, rely=0.6, anchor=CENTER, relwidth=0.9, relheight=0.7)
    scroll.grid_columnconfigure((0, 1, 2, 3, 4, 5), weight=1)
```

```
def editar_alunas(janela):
    academia = db.Academia()
    widgetEditor = ctk.CTkFrame(janela, fg_color="transparent", width=1450, height=750)
    widgetEditor.place(relx=0.5, rely=0.58, anchor=CENTER, relwidth=0.9, relheight=0.7)
    widgetEditor.grid_columnconfigure((0, 1, 2, 3, 4, 5), weight=1)

    label = ctk.CTkLabel(widgetEditor, text="Editar Aluna", font=("Segoe UI Black", 30))
    label.grid(row=0, column=0, columnspan=6, pady=20)

    labelInstrucao = ctk.CTkLabel(widgetEditor, text="Insira o nome ou CPF da aluna:", font=("Arial", 15))
    labelInstrucao.grid(row=1, column=1, padx=5, pady=10, sticky=E)
    entryEditar = ctk.CTkEntry(widgetEditor, font=("Arial", 15), width=300)
    entryEditar.grid(row=1, column=2, columnspan=2, pady=10)
    botaoPesquisar = ctk.CTkButton(widgetEditor, text="Buscar", font=("Arial", 15), command=lambda: buscar_alunas())
    botaoPesquisar.grid(row=1, column=4, pady=10, sticky=W)

    label_status = ctk.CTkLabel(widgetEditor, text="", font=("Arial", 15))
    entryacao = None
    alunaEdit = None
    cpf_para_editar = None
    boxInfo = None
    entry_nova_info = None
    switch_editar = {
        "Nome": "nome", "Data de Nascimento": "nascimento", "CEP": "cep", "Endereço": "endereco",
        "Bairro": "bairro", "Celular": "celular", "CPF": "cpf", "Quantidade de Dias": "dias",
        "Dias da Semana": "diasSemana", "Horário": "horario", "Valor da Mensalidade": "valor", "Vencimento": "vencimento"}

    scroll = ctk.CTkScrollableFrame(widgetEditor, fg_color="transparent")
    # de design de antes
```

Depois da modificação isso foi corrigido e não aparecia mais o scroll em outras abas:

```
def buscar_alunas():
    nonlocal entryacao

    for widget in widgetEditor.winfo_children():
        if isinstance(widget, ctk.CTkScrollableFrame):
            widget.destroy()

    label_status.configure(text="")
    label_status.grid_forget()

    scroll = ctk.CTkScrollableFrame(widgetEditor, fg_color="transparent")
    scroll.grid(row=2, column=0, columnspan=6, sticky="nsew", padx=20, pady=10)
    scroll.grid_columnconfigure((0, 1, 2, 3, 4, 5), weight=1)

    alunas = academia.buscarPorNome_ou_Cpf(entryEditor.get().strip())
```

```
def editar_alunas(janela):
    academia = db.Academia()
    widgetEditor = ctk.CTkFrame(janela, fg_color="transparent", width=1450, height=750)
    widgetEditor.place(relx=0.5, rely=0.58, anchor=CENTER, relwidth=0.9, relheight=0.7)
    widgetEditor.grid_columnconfigure((0, 1, 2, 3, 4, 5), weight=1)

    label = ctk.CTkLabel(widgetEditor, text="Editar Aluna", font=("Segoe UI Black", 30))
    label.grid(row=0, column=0, columnspan=6, pady=20)

    labelInstrucao = ctk.CTkLabel(widgetEditor, text="Insira o nome ou CPF da aluna:", font=("Arial", 15))
    labelInstrucao.grid(row=1, column=1, padx=5, pady=10, sticky=E)
    entryEditor = ctk.CTkEntry(widgetEditor, font=("Arial", 15), width=300)
    entryEditor.grid(row=1, column=2, columnspan=2, pady=10)
    botaoPesquisar = ctk.CTkButton(widgetEditor, text="Buscar", font=("Arial", 15), command=lambda: buscar_alunas())
    botaoPesquisar.grid(row=1, column=4, pady=10, sticky=W)

    label_status = ctk.CTkLabel(widgetEditor, text="", font=("Arial", 15))
    entryacao = None
    alunaEdit = None
    cpf_para_editar = None
    boxInfo = None
    entry_nova_info = None
    scroll = None
    switch_editar = {
        "Nome": "nome", "Data de Nascimento": "nascimento", "CEP": "cep", "Endereço": "endereco",
        "Bairro": "bairro", "Celular": "celular", "CPF": "cpf", "Quantidade de Dias": "dias",
        "Dias da Semana": "diasSemana", "Horário": "horario", "Valor da Mensalidade": "valor", "Vencimento": "vencimento"}
```

REFATORAÇÃO DE NOVAS FUNCIONALIDADES:

Cadastro de usuário e verificação de login:

Para o teste de cadastro de usuário, foi necessário cadastrar um nome e senha, antes de tudo. Depois, foi testado o login utilizando os dados previamente utilizados e inserções incorretas.

```

class Usuario:
    """
    Classe simples para representar usuário com nome e senha (armazenada como hash com salt).
    """
    def __init__(self, nome: str, pwd_hash: str = None, salt: str = None):
        self.nome = (nome or "").strip()
        self.pwd_hash = pwd_hash # hex string
        self.salt = salt # hex string

    def set_password(self, senha: str):
        """Gera salt curto e armazena o hash SHA256(salt + senha)."""
        if senha is None:
            raise ValueError("Senha não pode ser None")
        # salt simples suficiente para uso local; para produção use PBKDF2/etc.
        self.salt = secrets.token_hex(8) # 8 bytes -> 16 hex chars
        self.pwd_hash = hashlib.sha256((self.salt + senha).encode('utf-8')).hexdigest()

    def check_user(self, nome: str) -> bool:
        """Verifica se o user bate com o user armazenado."""
        if not (self.nome and self.pwd_hash and self.salt):
            return False
        return self.nome

    def check_password(self, senha: str) -> bool:
        """Verifica se a senha bate com o hash armazenado."""
        if not (self.pwd_hash and self.salt):
            return None
        tentativa = hashlib.sha256((self.salt + (senha or "")).encode('utf-8')).hexdigest()
        return self.pwd_hash == tentativa

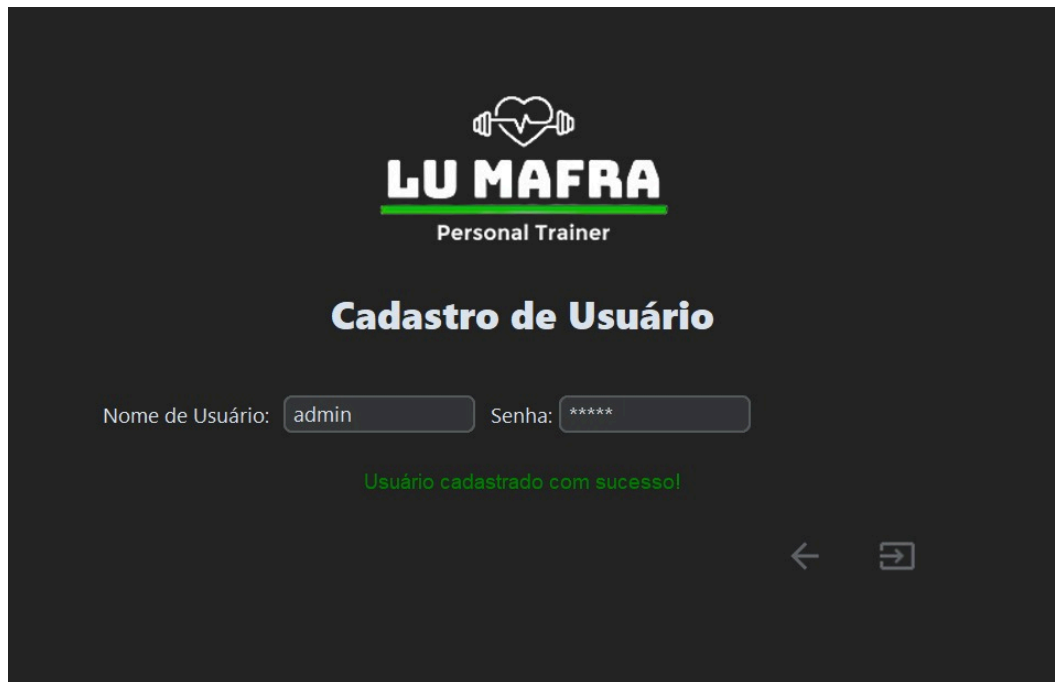
    def to_db_tuple(self) -> tuple:
        """(nome, pwd_hash, salt) - para INSERT no DB."""
        return (self.nome, self.pwd_hash, self.salt)

    @classmethod
    def from_db_row(cls, row: tuple):
        """Cria Usuário a partir de linha (id, nome, pwd_hash, salt) ou (nome, pwd_hash, salt)."""
        if not row:
            return None
        if len(row) == 4:
            _, nome, pwd_hash, salt = row
        elif len(row) == 3:
            nome, pwd_hash, salt = row
        else:
            raise ValueError("Formato de row inválido")
        return cls(nome=nome, pwd_hash=pwd_hash, salt=salt)

```

INTEGRAÇÃO DE FUNCIONALIDADES DO T2 + NOVAS:

Como dito anteriormente a nova funcionalidade foi modificada para não precisar de email, então durante a parte gráfica houve a remoção do email e integrada no sistema



REFATORAÇÕES GERAIS NO CÓDIGO:

Como última modificação, fizemos um ajuste completo na área de edição de alunas. Este aprimoramento teve como objetivo deixar o código mais eficiente, claro e funcional, reduzindo a possibilidade de falhas durante a execução. Durante esse processo, foram revisadas estruturas, corrigidos comportamentos inesperados e otimizada a lógica utilizada, garantindo melhor desempenho e maior estabilidade da aplicação.

Além disso, após a implementação dessas melhorias, toda a equipe realizou uma verificação geral no código, analisando cada parte cuidadosamente para identificar e eliminar possíveis erros que sobraram. Essa etapa final de verificação foi essencial para assegurar que o sistema estivesse funcionando corretamente e pronto para ser entregue.

Em resumo, o último ajuste feito teve como principal tópico a otimização da área de edição das alunas, seguido por uma revisão completa do projeto para garantir sua finalização sem bugs e com total confiabilidade.



LU MAFRA

Personal Trainer



Editar Aluna

Selecione o campo para editar de marca:

Data de Nascimento



Editar

2110102929